

# Short Write-Ups (No Coding)

C# Fundamentals - Delegates, var, and Anonymous Types

## ***1. In your own words, what is the difference between a delegate and a lambda expression?***

A **delegate** is a **type** - it's the definition of a method signature (what parameters it takes and what it returns), acting as a reference type that can point to any method matching that signature. A **lambda expression** is just one convenient way to **create a value** of a delegate type - an inline, unnamed function that gets assigned to a delegate variable. In short: the delegate is the "box" (the contract/shape a callable thing must fit), and a lambda is one quick way to fill that box with actual behavior, instead of writing a separate named method and passing its name.

## ***2. Why is var still statically typed even though the type isn't written explicitly?***

var doesn't mean "no type" or "decide the type at runtime" - it just tells the compiler to **infer** the variable's type automatically from whatever is assigned to it on the right-hand side, at **compile time**. Once that type is inferred (e.g., var x = 5; makes x an int, var name = "Adham"; makes name a string), it's fixed for that variable exactly as if it had been written explicitly - you can't later assign a value of a different type to it, and the compiler still performs all the same type checking it normally would. var is purely a syntax convenience for the developer; it has zero effect on how strongly and statically typed the resulting code actually is.

## ***3. Give one real scenario where an anonymous type is more convenient than creating a full class.***

A common scenario is shaping data for a quick, temporary result - for example, using LINQ to project only a few fields from a larger object for display purposes:

```
var summary = employees.Select(e => new { e.Name, e.Salary });
```

Here, you only need a lightweight, throwaway object holding just Name and Salary - creating a whole named class (e.g., EmployeeSummaryDto) just for this one local, short-lived use would be unnecessary overhead and clutter in the codebase. An anonymous type gives you a quick, strongly-typed (yet unnamed) object for exactly this kind of temporary, local data-shaping need, without polluting the project with extra class definitions that have no reuse value elsewhere.